SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I ***Chandra Prakash D / 192524060*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

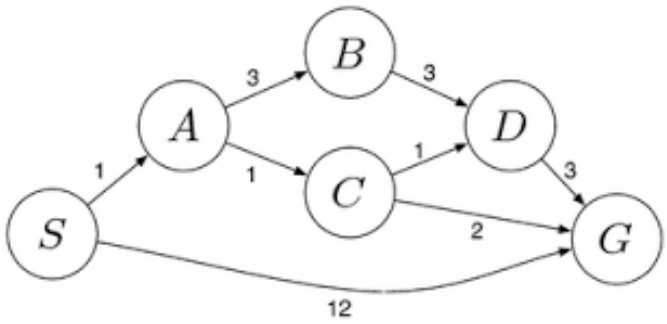
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

ANSWER SHEET

Artificial Intelligence (CSA17) – Assessment Tool 1: Analytical Problem Solving

Name	Chandra Prakash D
Register No.	192524060
Course	Artificial Intelligence (CSA17) – SIMATS Engineering
Course Outcome	CO1 – Intelligent agents and uninformed search methods (BL3)

Question 1: Water Jug Problem

1.1 Problem formulation

Let **x** = gallons in the 4-gallon jug ($0 \le x \le 4$) and **y** = gallons in the 3-gallon jug ($0 \le y \le 3$). A state is the ordered pair (x, y); the pair is written **(x, y)** below.

State space	All pairs (x, y) with $x \in \{0,\ldots,4\}$, $y \in \{0,\ldots,3\}$ – 20 states (14 are reachable)
Initial state	(0, 0) – both jugs empty
Goal test	x = 2 (exactly 2 gallons in the 4-gallon jug; y may be anything)
Actions	Fill, empty, or pour (listed in the rule table below)
Path cost	1 per action (so the best solution uses the fewest moves)

1.2 Production rules (operators)

No.	Condition	Resulting state	Meaning
1	$x < 4$	(4, y)	Fill the 4-gallon jug from the pump
2	$y < 3$	(x, 3)	Fill the 3-gallon jug from the pump
3	$x > 0$	(0, y)	Empty the 4-gallon jug on the ground
4	$y > 0$	(x, 0)	Empty the 3-gallon jug on the ground
5	$x + y \ge 4$ and $y > 0$	$(4, y - (4 - x))$	Pour 3-gal into 4-gal until 4-gal is full
6	$x + y \ge 3$ and $x > 0$	$(x - (3 - y), 3)$	Pour 4-gal into 3-gal until 3-gal is full
7	$x + y \le 4$ and $y > 0$	$(x + y, 0)$	Pour all of 3-gal into 4-gal
8	$x + y \le 3$ and $x > 0$	$(0, x + y)$	Pour all of 4-gal into 3-gal

1.3 Solution (found by Breadth-First Search)

Searching level by level from (0, 0) with the rules above reaches the goal first at depth 6, so **6 moves is the minimum**. The solution path is:

Step	Rule applied	State (x, y)	Action performed
0	–	(0, 0)	Start: both jugs empty
1	Rule 1	(4, 0)	Fill the 4-gallon jug from the pump
2	Rule 6	(1, 3)	Pour from the 4-gal jug into the 3-gal jug until it is full (1 gallon stays behind)
3	Rule 4	(1, 0)	Empty the 3-gallon jug on the ground
4	Rule 8	(0, 1)	Pour the 1 gallon from the 4-gal jug into the 3-gal jug
5	Rule 1	(4, 1)	Fill the 4-gallon jug from the pump again
6	Rule 6	(2, 3)	Pour from the 4-gal jug into the 3-gal jug (needs 2 more gallons) → 2 gallons remain in the 4-gal jug ✓ GOAL

Path: (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3)

Alternative 6-move solution: (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0), using rules 2, 7, 2, 5, 3, 7.

Result: Exactly 2 gallons of water can be measured into the 4-gallon jug in 6 steps.

Question 2: Mars Rover – Intelligent Agent Analysis

A Mars rover (for example NASA's Curiosity/Perseverance) is a mobile robotic agent that drives across the Martian surface, chooses interesting rocks and soil, collects them, analyses them with onboard instruments, and sends the results to Earth. It is described below using the PEAS framework (Performance measure, Environment, Actuators, Sensors).

(i) Percepts of the agent

Percepts are the inputs the rover receives from its sensors and from Earth:

- **Camera images:** stereo navigation cameras, mast camera, hazard-avoidance cameras (terrain, rocks, obstacles, slopes).
- **Spectrometer and chemical-sensor readings:** mineral and chemical composition of rocks, soil and atmosphere, plus traces of water or organic compounds.
- **Contact and drill sensors:** force/torque on the arm and drill, and whether a sample was actually collected.
- **Navigation and state sensors:** wheel encoders, gyroscope/IMU, tilt (slope) sensor, sun sensor and wheel-slip readings.
- **Environmental sensors:** temperature, wind, dust and radiation levels.
- **Internal health sensors:** battery charge, solar or RTG power output, motor currents and component temperatures.
- **Communication percepts:** commands, schedules and updated targets uplinked from Earth, and link availability (orbiter relay windows).

(ii) Characterisation of the operating environment

Property	Value	Reason
Observable	Partially observable	Cameras and sensors see only a limited range; terrain behind rocks, underground layers and soil strength are hidden.

Property	Value	Reason
Agents	Single agent	Only the rover acts on the surface (other rovers or orbiters do not interfere with its actions).
Deterministic?	Stochastic	Wheel slip, sand, dust storms and sensor noise mean the same action does not always have the same outcome.
Episodic?	Sequential	Present decisions (path, battery use, sample choices) affect future options and the whole mission.
Static?	Dynamic	Wind, dust, temperature, lighting and the rover's own power level keep changing while it decides.
Discrete?	Continuous	Position, speed, arm angles, temperature and time are continuous values.
Known?	Largely unknown	The terrain is only partly mapped, so the rover must explore and learn.
Other	Remote, with delay	Earth–Mars signal delay is about 4–24 minutes one way, so real-time remote control is impossible and the rover must act autonomously.

(iii) Actions the agent can take (actuators)

- **Mobility:** drive forward/backward, turn, steer, stop, climb, back off from hazards, reorient toward the Sun.
- **Sampling:** deploy the robotic arm, scoop soil, drill or abrade rock, place instruments on the target, store the sample in the analysis unit.
- **Analysis:** switch on and calibrate the spectrometer or camera, run the analysis, discard or keep the sample.
- **Imaging:** pan and tilt the mast and capture photographs.
- **Communication:** compress data, transmit to Earth or an orbiter, and receive uplinked commands.
- **Self-maintenance:** charge batteries, switch heaters on, go into sleep or safe mode, and clean or stow the arm.

(iv) Performance measure

Criterion	How it is measured
Scientific value	Number and quality of samples analysed; scientific importance of the sites (signs of water, organics, interesting minerals).
Coverage / exploration	Distance travelled and area explored per unit time or energy.
Safety and survival	No collisions, no falls or getting stuck, no damage to the instruments; mission lifetime in sols (Martian days).
Energy efficiency	Power used per task; battery never dropping below the safe level.
Data return	Amount of useful, correct data successfully transmitted to Earth.
Autonomy and time	Tasks completed per sol with minimal commands from the ground.

A weighted utility score such as $U = w_1 \cdot (\text{science value}) + w_2 \cdot (\text{coverage}) + w_3 \cdot (\text{data returned}) - w_4 \cdot (\text{energy used}) - w_5 \cdot (\text{risk / damage})$ can combine these into one number to compare different plans.

(v) Most suitable agent architecture and why

Best choice: a hybrid, layered agent whose core is a utility-based (goal-based) agent with a model, plus learning and a model-based reflex layer for safety.

- **Reflex layer (bottom):** immediate condition–action rules for emergencies (tilt too high → stop; battery low → sleep/charge; obstacle very close → halt). They must react in milliseconds without waiting for Earth.
- **Model-based layer:** the environment is only partially observable, so the rover keeps an internal map and state (position, terrain, battery, which samples are done) and updates it from percepts.
- **Goal-based layer:** the rover has explicit goals (reach a target rock, collect a sample, transmit data), and plans paths with search algorithms such as A*.
- **Utility-based layer:** many goals compete (science vs. energy vs. risk), so a utility function chooses the best trade-off rather than just any goal-reaching action.
- **Learning component:** improves terrain classification and picks better science targets as it gathers experience on a new planet.

A pure simple-reflex agent would fail because the environment is partially observable and the tasks need planning, while a pure planner would be too slow for hazards. The signal delay to Earth rules out remote control, so a layered autonomous architecture is the most suitable.

Question 3: 8-Queens Problem – Formulation and Search

The goal is to place 8 queens on an 8 × 8 chessboard so that no two queens attack each other, i.e. no two share a row, a column or a diagonal. In the configuration given in the question the queen in column 1 and the queen in column 8 lie on the same diagonal, so the board is not a solution. The figure below shows that configuration (rows numbered from the top for columns 1 to 8: 1, 3, 5, 7, 2, 4, 6, 8) and a valid goal state for comparison.

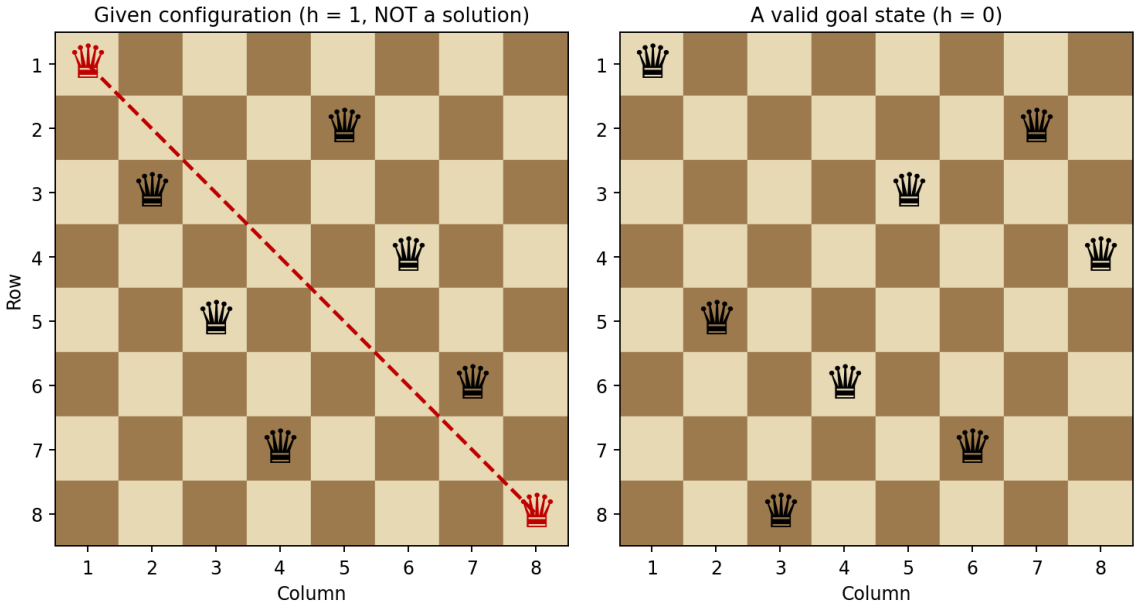


Figure: *Left – the given configuration has exactly one attacking pair (columns 1 and 8, red dashed line), so h = 1. Right – a goal state with no attacking pairs (rows 1, 5, 8, 6, 3, 7, 2, 4).*

3.1 Problem components

(a) Incremental formulation (best for systematic search)

States	Any arrangement of 0 to 8 queens on the board in which no queen attacks another. Queens are added one column at a time from the left, so at most one queen per column.
Initial state	Empty board (no queens).
Actions	Place a queen in a square of the leftmost empty column that is not attacked by any queen already on the board.
Transition model	Returns the board with the new queen added.
Goal test	8 queens are on the board and none is attacked.
Path cost	Not important – only the final arrangement matters (each step costs 1, and every solution is at depth 8).

(b) Complete-state formulation (for local search)

States	All 8 queens on the board, one per column (8^8 = 16,777,216 arrangements).
Initial state	Any random arrangement, e.g. the configuration in the figure.
Actions	Move one queen to another square in its own column.
Heuristic h	Number of pairs of queens that attack each other (the figure has h = 1).
Goal test	h = 0.

3.2 Size of the search space

Formulation	Number of sequences / states
Naive: place any of the 8 queens on any free square (64 × 63 × … × 57)	$\approx 1.8 \times 10^{14}$
One queen per column (any row) – 8^8	16,777,216
Incremental with “not attacked” rule	only 2,057 states

The problem has **92 solutions** in total (12 distinct ones if rotations and reflections are treated as the same). The 2,057-state incremental formulation shows how a good formulation shrinks the search space dramatically.

3.3 Search strategies applied

Strategy	Behaviour on 8-Queens
Breadth-First Search	Expands all partial boards level by level. Complete, but every solution lies at depth 8, so it must store a very large frontier (high memory).
Depth-First Search / Backtracking	Place a queen in column 1, then column 2, and so on. If a column has no safe row, backtrack to the previous column and try the next row. Needs memory for only one path (depth 8), so it is the standard systematic method. The first solution found is rows 1, 5, 8, 6, 3, 7, 2, 4.
Depth-Limited / Iterative Deepening	Depth is known to be exactly 8, so a limit of 8 is ideal; iterative deepening gives BFS-like completeness with DFS memory.

Strategy	Behaviour on 8-Queens
Hill Climbing (local search)	Starts from a full board (like the figure) and moves a queen to lower h. It can get stuck on local minima or plateaus (a steepest-ascent run solves about 14 % of random starts), so random restarts are used.
Min-Conflicts / Simulated Annealing	Pick a conflicted queen and move it to the row with fewest conflicts. Solves even very large n-Queens problems almost instantly.

Conclusion: With the incremental formulation, depth-first backtracking is the best uninformed method because the solution depth is fixed and small. For the given board (h = 1), min-conflicts local search is the quickest way to repair it.

Question 4: OLA Cab Booking – Problem-Solving Agent

4.1 Type of agent

The suitable agent is a **goal-based problem-solving agent** with a utility-based component for choosing the cab type. It first formulates a goal (reach the destination), then formulates the problem (states, actions, goal test, cost), then searches for a sequence of actions, and finally executes it. The environment is observable (the app shows locations, fares and ETAs), single-agent, sequential and mostly deterministic, so offline planning of this kind works well.

Goal	Customer reaches destination D from pickup point P.
States	(location, selected cab type, booking status, ride progress).
Initial state	Customer at P, no cab booked.
Actions	Choose cab type (Micro, Mini, Prime Sedan, Share…), confirm booking, wait for driver, ride along a road segment, pay.
Goal test	Current location = D and the ride is completed.
Path cost	Fare + travel time (+ comfort penalty), weighted by the customer's preference.

4.2 General problem-solving agent (pseudocode)

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq     // action sequence, initially empty
              state   // description of the current world state
              goal    // goal, initially null
              problem // problem formulation


  state <- UPDATE-STATE(state, percept)     // GPS location, cab list, fares
  if seq is empty then
     goal    <- FORMULATE-GOAL(state)      // be at destination D
     problem <- FORMULATE-PROBLEM(state, goal)
     seq     <- SEARCH(problem)
     if seq = failure then return a null action
  action <- FIRST(seq)
  seq    <- REST(seq)
  return action
```

4.3 Pseudocode specific to the OLA application

```
Algorithm OLA-CAB-BOOKING
Input : pickup P, destination D, preferences (budget, comfort, max_wait)
Output: completed ride or failure message

1.  START
2.  goal <- D
3.  cabs <- GET-AVAILABLE-CABS(P)   // Micro, Mini, Sedan, Share...
4.  if cabs is empty then DISPLAY("No cabs available"); STOP
5.  route <- UNIFORM-COST-SEARCH(roadGraph, P, D)   // or A*
6.  for each cab c in cabs do
       c.fare <- ESTIMATE-FARE(c.type, route.distance, route.time)
       c.eta  <- ESTIMATE-ARRIVAL(c, P)
       c.util <- w1*c.comfort - w2*c.fare - w3*c.eta
    end for
7.  DISPLAY cabs sorted by c.util
8.  selected <- customer's choice (default: cab with highest c.util)
9.  CONFIRM-BOOKING(selected, P, D)
10. repeat
       driver <- ASSIGN-DRIVER(selected)
       if driver = none then
          selected <- next best cab in cabs  // fall back
    until driver is assigned or cabs exhausted
11. if no driver found then DISPLAY("Try again later"); STOP
12. TRACK driver until pickup at P; START-RIDE
13. while current_location != D do
       FOLLOW(route)
       if traffic or road block then route <- RE-PLAN(current_location, D)
    end while
14. END-RIDE; CALCULATE-FINAL-FARE; PAYMENT; RATING
15. STOP
```

Result: A goal-based problem-solving agent (with a utility function for cab choice and a UCS/A* search for the route) can complete the booking process from P to D.

Question 5: Uniform Cost Search on the Delivery Network

5.1 Problem understanding

Find the least-cost path from the start warehouse S to the goal warehouse G in the weighted directed graph. Edge costs (fuel + time) read from the figure:

S → A = 1	S → G = 12	A → B = 3	A → C = 1	B → D = 3
C → D = 1	C → G = 2	D → G = 3		

5.2 Method: Uniform Cost Search

UCS keeps a priority queue (frontier) ordered by the path cost g(n) from S. It always expands the lowest-cost node, and the goal test is applied when a node is *removed* from the queue (not when generated). If a cheaper route to a frontier node is found, its cost is updated. This guarantees the first time G is removed, the path is optimal because all costs are positive.

```
function UNIFORM-COST-SEARCH(problem) returns path or failure
  frontier <- priority queue ordered by g; insert (S, g=0)
  explored <- empty set
  loop
    if frontier is empty then return failure
    node <- POP(frontier)              // lowest g
```

```
if node = G then return PATH(node)  // goal test on removal
add node to explored
for each edge (node -> child, cost) do
  g' <- node.g + cost
  if child not in explored and not in frontier then
    INSERT(child, g')
  else if child in frontier with higher g then
    REPLACE its cost with g'
```

5.3 Step-by-step trace

Step	Node removed (g)	Frontier after expansion [node : cost via parent]
0	–	S : 0
1	S (0)	A : 1 (S), G : 12 (S)
2	A (1)	C : 2 (A), B : 4 (A), G : 12 (S)
3	C (2)	D : 3 (C), B : 4 (A), G : 4 (C) [improves 12 → 4]
4	D (3)	B : 4 (A), G : 4 (C) [G via D = 3 + 3 = 6 > 4, rejected]
5	B (4)	G : 4 (C) [D via B = 4 + 3 = 7 > 3 and D already explored, ignored]
6	G (4)	Goal removed → stop

Order of expansion: S → A → C → D → B → G (the cost-4 tie between B and G is broken first-in-first-out).

5.4 Comparison of all paths from S to G

Path	Total cost	Remark
S → A → C → G	1 + 1 + 2 = 4	Least cost (optimal)
S → A → C → D → G	1 + 1 + 1 + 3 = 6	Higher cost
S → A → B → D → G	1 + 3 + 3 + 3 = 10	Higher cost
S → G	12	Fewest edges but most expensive

5.5 Result and interpretation

Optimal path: S → A → C → G with a minimum total cost of 4 units (fuel + time).

- The direct route S → G looks shortest in number of edges (it is what Breadth-First Search would return), but it costs 12, three times more than the UCS answer.
- UCS is complete and optimal because every edge cost is positive. Its time and space complexity is O(b^(1 + ⌊C*/ε⌋)), where C* is the optimal cost (4) and ε the smallest edge cost (1).
- Only 5 nodes (S, A, C, D, B) were expanded; UCS has no heuristic, so it also expands off-path nodes such as B whose cost is not above the goal's cost.
- Routing packages via warehouses A and C saves 8 cost units (about 67 %) versus the direct route.